

Question 1 - Annapurna Stores

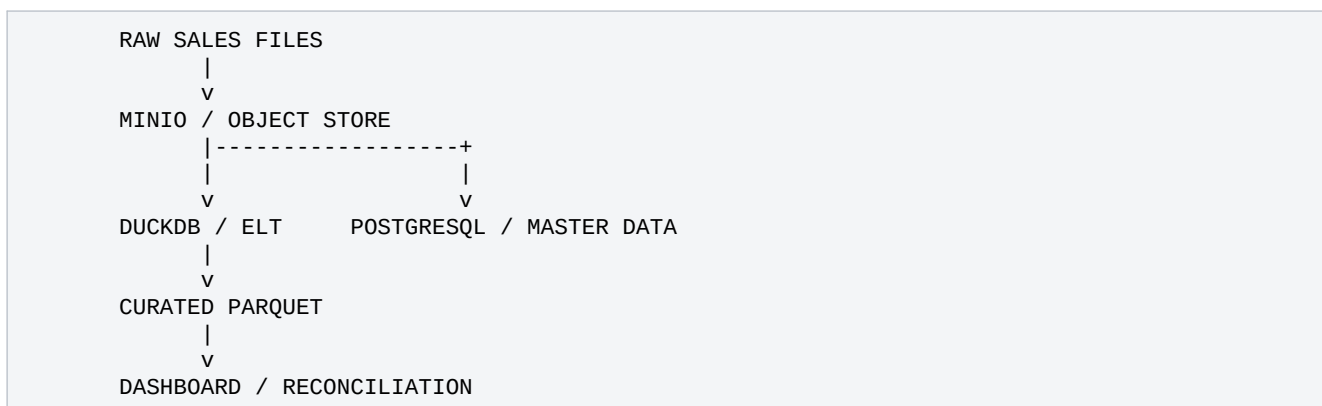
Student Solution - Modern Data Platform

Item	Result
Source files	4,457
Raw manifest rows	1,137,585
Deduplicated fact lines	1,120,924
Source net revenue	INR 522.87M

Objective

Build a repeatable analytical platform so that October is always October, revenue can be sliced by store, product category, day of week and month, and historical prices are returned according to the reporting period.

Architecture



Key design points

- Business month comes from the file contract / bill date used for reporting.
- Raw deliveries are kept, but duplicate logical lines are blocked in the analytical fact table.
- Revenue includes SALE, RETURN, DISCOUNT and VOID; TAX and TENDER do not contribute to merchandise revenue.
- Product identity uses the durable product key and validity dates because product codes were reissued.
- Historical price is resolved with an as-of temporal join.

A. Stand the platform up and land the data

Use MinIO for object storage, PostgreSQL for master data, and DuckDB for analytical processing. Keep the vendor files unchanged in the raw zone and use a partitioned curated zone for reporting.

Object-store layout

```
raw/sales/store_id=S03/business_date=2024-10-14/SALES_S03_20241014.csv
curated/sales/store_id=S03/business_month=2024-10/part-000.parquet
```

The curated layout is partitioned by store and business month. A query for one store and one month can therefore prune unrelated stores and months.

S03 / October 2024	Files	Bytes	Rows
Flat folder	4,457	68,706,877	1,137,585
Raw store-month	31	872,458	13,924
Curated store-month	1	163,080	13,924

Measured S03 October net revenue = **6,525,145.39**.

Working query

```
SELECT store_id, business_month,
       SUM(CASE WHEN line_type IN ('SALE', 'RETURN', 'DISCOUNT', 'VOID')
              THEN qty * unit_price ELSE 0 END) AS net_revenue
FROM read_parquet(
    'curated/sales/store_id=S03/business_month=2024-10/*.parquet',
    hive_partitioning=true)
GROUP BY 1, 2;
```

Measurement commands

```
$ find data/sales -maxdepth 1 -type f -printf '%s\n' | awk '{bytes+=$1} END{print "files="NR, "bytes="bytes}'
files=4457 bytes=68706877

$ find data/sales -maxdepth 1 -name 'SALES_S03_202410*.csv' -type f | wc -l
31

$ find data/sales -maxdepth 1 -name 'SALES_S03_202410*.csv' -type f -printf '%s\n' | awk '{b+=$1} END{print b}'
872458
```

B. Make the load safe to run twice

The loader uses conflict-safe insertion on the logical line key. Replaying the same delivery does not create another analytical line.

```
INSERT INTO fact_sales_line (...)  
VALUES (...)  
ON CONFLICT (bill_no, line_no) DO NOTHING;
```

Run	Rows before	Inserted	Final rows	SHA-256
1	0	1,120,924	1,120,924	b9ecf1abad82f55ae0ec0d432cc5cca5d8f28c19d3fe9089ee523ec5453d7824
2	1,120,924	0	1,120,924	b9ecf1abad82f55ae0ec0d432cc5cca5d8f28c19d3fe9089ee523ec5453d7824
3	1,120,924	0	1,120,924	b9ecf1abad82f55ae0ec0d432cc5cca5d8f28c19d3fe9089ee523ec5453d7824

Duplicate resend rows = **16,661**; conflicting duplicate keys = **0**. All three runs produce the same final row count and checksum.

Verifier output

```
SOURCE_FILES=4457  
SOURCE_ROWS=1137585  
UNIQUE_LINE_KEYS=1120924  
DUPLICATE_ROWS=16661  
CONFLICTING_DUPLICATE_KEYS=0  
TIMESTAMP_ROWS_CHECKED=1120924  
TIMESTAMP_BUSINESS_DATE_MISMATCHES=9272  
EXPECTED_CHECKSUM=b9ecf1abad82f55ae0ec0d432cc5cca5d8f28c19d3fe9089ee523ec5453d7824  
RUN_1: before=0 inserted=1120924 rows=1120924 checksum=b9ecf1abad82f55ae0ec0d432cc5cca5d8f28c19d3fe9089ee523ec5453d7824  
RUN_2: before=1120924 inserted=0 rows=1120924 checksum=b9ecf1abad82f55ae0ec0d432cc5cca5d8f28c19d3fe9089ee523ec5453d7824  
RUN_3: before=1120924 inserted=0 rows=1120924 checksum=b9ecf1abad82f55ae0ec0d432cc5cca5d8f28c19d3fe9089ee523ec5453d7824
```

C. Design the table behind the dashboard

Fact grain: one row in the analytical fact table = one transaction line on one bill. SALE, RETURN, DISCOUNT, VOID, TAX and TENDER remain as lines; line type controls revenue contribution.

Table	Purpose
dim_date	Calendar date, weekday and month
dim_store	Store identity and address attributes
dim_category	Category and department
dim_product	Durable product key, code, name and validity interval
fact_sales_line	Bill/line, date, store, product_sk, line_type, qty, unit_price, amount

Product code is not treated as the durable identity because 24 codes were reissued. Store/product descriptions stay in dimensions instead of repeating them on every fact row.

D. Historical prices without changing code

Use one parameterized as-of query. Only :report_period changes.

```
WITH params AS (SELECT CAST(:report_period AS DATE) AS as_of)
SELECT p.product_code, p.product_name, params.as_of,
       pr.mrp, pr.selling_price
FROM dim_product p
CROSS JOIN params
JOIN price_revisions pr
  ON pr.product_sk = p.product_sk
  AND params.as_of BETWEEN pr.effective_from AND pr.effective_to
WHERE p.product_code = :product_code;
```

Product	Period	MRP	Selling price
P100049 - Britannia Cream Biscuit 60g	2024-03-01	74.98	66.95
P100049 - Britannia Cream Biscuit 60g	2024-10-01	77.20	68.93

The same query returns the correct price for different reporting periods without changing SQL. PostgreSQL contains 1,224 products and 4,320 price revisions; 24 product codes are reissued.

E. Query across the two systems

Sales remain in Parquet while product/category masters remain in PostgreSQL. DuckDB reads both and performs the joins without first copying either source into the other.

```
WITH sales AS (
  SELECT product_code, CAST(business_date AS DATE) AS business_date,
         CAST(qty AS DOUBLE) AS qty, CAST(unit_price AS DOUBLE) AS unit_price,
         line_type
  FROM read_parquet('lake/store_id=S03/business_month=2024-10/*.parquet',
                    hive_partitioning=true)
  WHERE line_type IN ('SALE', 'RETURN', 'DISCOUNT', 'VOID')
), products AS (
  SELECT product_sk, product_code, category_id, valid_from, valid_to
  FROM postgres_scan('host=tmp port=55434 dbname=q1masters user=runner', 'public', 'products')
), categories AS (
  SELECT category_id, category_name
  FROM postgres_scan('host=tmp port=55434 dbname=q1masters user=runner', 'public', 'product_categories')
)
SELECT c.category_name, ROUND(SUM(s.qty * s.unit_price),2) AS net_revenue
FROM sales s
JOIN products p ON p.product_code=s.product_code
                  AND s.business_date BETWEEN p.valid_from AND p.valid_to
JOIN categories c ON c.category_id=p.category_id
GROUP BY c.category_name
ORDER BY net_revenue DESC;
```

Category	Net revenue	Lines
Staples & Grains	1,220,174.16	752
Baby Care	1,111,494.45	661
Edible Oils	971,631.57	675
Dairy	496,511.45	741
Personal Care	459,210.46	574

Observed operators: READ_PARQUET, POSTGRES_SCAN, HASH_JOIN and HASH_GROUP_BY. The plan proves federation and shows where the source reads and analytical operators were evaluated. No undocumented temporal-join pushdown is claimed.

F. Reconcile with finance_monthly.csv

Month	Source	Finance	Difference	Diagnosis / action
2024-01	38,446,071.33	38,446,071.33	0.00	Match - no action
2024-02	34,887,085.55	34,887,085.55	0.00	Match - no action
2024-03	41,971,649.09	42,457,899.09	486,250.00	Scope difference - institutional invoice outside till; take scope bridge to Finance
2024-04	37,958,457.37	37,958,457.37	0.00	Match - no action
2024-05	41,764,716.40	41,764,716.40	0.00	Match - no action
2024-06	38,987,082.82	38,987,082.82	0.00	Match - no action
2024-07	40,295,160.11	40,527,291.81	232,131.70	Source gap - S07 missing Jul 9-11; report gap to Finance
2024-08	45,252,181.75	45,252,181.75	0.00	Match - no action
2024-09	44,615,037.46	44,615,037.46	0.00	Match - no action
2024-10	56,359,195.92	56,359,195.92	0.00	Match - no action
2024-11	51,583,838.47	51,583,838.47	0.00	Match - no action
2024-12	50,745,259.48	50,745,209.00	-50.48	Rounding-definition difference - align bill-rounding policy with Finance

Conclusion: No month indicates a pipeline bug.

October control check

Summing every line type in the October source produces **122,501,668.06**, while correct merchandise net revenue is **56,359,195.92**. The excess is primarily TAX + TENDER. Duplicate/resent handling is the other major control.

Actions for exceptions

- March: take the institutional-invoice scope bridge to Finance.
- July: report the missing S07 July 9-11 source gap; do not invent transactions.
- December: align bill-rounding policy with Finance.

Evidence Appendix - Commands and outputs

1. Manifest count

```
$ awk -F, 'NR>1{files++; rows+=$4} END{print "files="files, "manifest_rows="rows}' data/_truth/file_manifest.csv
files=4457 manifest_rows=1137585
```

2. Raw byte count

```
$ find data/sales -maxdepth 1 -type f -printf '%s\n' | awk '{bytes+=$1} END{print "files="NR, "bytes="bytes}'
files=4457 bytes=68706877
```

3. S03 October file count and bytes

```
$ find data/sales -maxdepth 1 -name 'SALES_S03_202410*.csv' -type f | wc -l
31
$ find data/sales -maxdepth 1 -name 'SALES_S03_202410*.csv' -type f -printf '%s\n' | awk '{b+=$1} END{print b}'
872458
```

4. Timestamp/business-date mismatch evidence

```
$ python q1_solution/scripts/verify_q1.py data | grep TIMESTAMP
TIMESTAMP_ROWS_CHECKED=1120924
TIMESTAMP_BUSINESS_DATE_MISMATCHES=9272
```

5. PostgreSQL master load

```
$ psql ... -f data/masters.sql
$ psql ... -Atc "select table_name,count(*) ..."
stores=12
product_categories=14
products=1224
price_revisions=4320
reissued_product_codes=24
```

Evidence Appendix - DuckDB execution plan

Raw plan output used to identify where federation and analytical operators were evaluated.

```
lake_rows 1120924
lake_files 144
s03_oct_objects 1
s03_oct_bytes 163080
partition_query [(S03, 2024-10, 13924, 6525145.39)]

TOP_N
  PROJECTION
    HASH_GROUP_BY
      PROJECTION
        HASH_JOIN
          PROJECTION
            POSTGRES_SCAN Table: products
          HASH_JOIN
            POSTGRES_SCAN Table: product_categories
```

Evidence Appendix - DuckDB execution plan (continued)

```

FILTER
  ((line_type = SALE) OR
   (line_type = RETURN) OR
   (line_type = DISCOUNT) OR
   (line_type = VOID))

READ_PARQUET
  Function: READ_PARQUET
  Projections: line_type, product_code,
               business_date, qty, unit_price
  Filters: optional line_type IN
           (SALE, RETURN, DISCOUNT, VOID)
  Estimated rows: ~13,924

POSTGRES_SCAN
  Table: products
  Projections: product_code, category_id, valid_from, valid_to

POSTGRES_SCAN
  Table: product_categories
  Projections: category_id, category_name
```

Interpretation

- READ_PARQUET = DuckDB evaluated the object-store side.
- POSTGRES_SCAN = DuckDB read the PostgreSQL master tables.
- HASH_JOIN and HASH_GROUP_BY = DuckDB performed the join and aggregation.
- The plan shows the merchandise-line filter on the Parquet side; it does not show the temporal join predicate being pushed into PostgreSQL.

SQL Appendix - Star schema

```
CREATE TABLE dim_date (  
    date_key INTEGER PRIMARY KEY,  
    calendar_date DATE NOT NULL UNIQUE,  
    day_of_week SMALLINT NOT NULL,  
    day_name TEXT NOT NULL,  
    month_key INTEGER NOT NULL,  
    month_start DATE NOT NULL,  
    month_name TEXT NOT NULL  
);  
  
CREATE TABLE dim_store (  
    store_sk BIGSERIAL PRIMARY KEY,  
    store_id TEXT NOT NULL UNIQUE,  
    store_name TEXT NOT NULL,  
    address_line TEXT NOT NULL,  
    city TEXT NOT NULL,  
    state TEXT NOT NULL,  
    region TEXT NOT NULL,  
    floor_area_sqft INTEGER NOT NULL,  
    opened_on DATE NOT NULL  
);  
  
CREATE TABLE dim_category (  
    category_sk BIGSERIAL PRIMARY KEY,  
    category_id TEXT NOT NULL UNIQUE,  
    category_name TEXT NOT NULL,  
    department TEXT NOT NULL,  
    gst_rate NUMERIC(4,3) NOT NULL  
);  
  
CREATE TABLE dim_product (  
    product_sk BIGINT PRIMARY KEY,  
    product_code TEXT NOT NULL,  
    product_name TEXT NOT NULL,  
    category_id TEXT NOT NULL,  
    valid_from DATE NOT NULL,  
    valid_to DATE NOT NULL,  
    is_current BOOLEAN NOT NULL  
);
```

SQL Appendix - Fact table and analytical queries

```
CREATE TABLE fact_sales_line (
  bill_no TEXT NOT NULL,
  line_no INTEGER NOT NULL,
  business_date DATE NOT NULL,
  store_id TEXT NOT NULL,
  product_sk BIGINT,
  product_code TEXT,
  line_type TEXT NOT NULL,
  qty NUMERIC(18,3) NOT NULL,
  unit_price NUMERIC(18,2) NOT NULL,
  amount NUMERIC(18,2) GENERATED ALWAYS AS (qty * unit_price) STORED,
  source_file TEXT NOT NULL,
  loaded_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (bill_no, line_no),
  CHECK (line_type IN ('SALE','RETURN','DISCOUNT','VOID','TAX','TENDER'))
);

CREATE INDEX ix_fact_sales_month_store
  ON fact_sales_line (business_date, store_id);
CREATE INDEX ix_fact_sales_product_date
  ON fact_sales_line (product_code, business_date);
CREATE INDEX ix_fact_sales_revenue
  ON fact_sales_line (business_date, line_type)
  WHERE line_type IN ('SALE','RETURN','DISCOUNT','VOID');
```

Dashboard query

```
SELECT f.store_id, c.category_name, d.day_name, d.month_start,
       SUM(f.amount) AS net_revenue
FROM fact_sales_line f
JOIN dim_date d ON d.calendar_date=f.business_date
JOIN dim_product p ON p.product_sk=f.product_sk
JOIN dim_category c ON c.category_id=p.category_id
WHERE f.line_type IN ('SALE','RETURN','DISCOUNT','VOID')
GROUP BY f.store_id, c.category_name, d.day_name, d.month_start;
```

Submission checklist

- MinIO + PostgreSQL + DuckDB platform is running.
- Sales data is organized by store and business month in the curated layer.
- Three-run idempotency proof has identical row counts and full checksums.
- Star schema uses an explicit fact grain and durable product identity.
- Historical prices use effective-date intervals.
- Federation is demonstrated with DuckDB execution-plan evidence.
- Finance exceptions are documented rather than forced to zero.